

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 5

Operating Systems

The File System with Your Own Hands

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment and evidence 3

2.2 Part 1 - The tree and file types 3

2.3 Part 2 - Inodes and links 4

2.4 Part 3 - Permissions 6

2.5 Part 4 - Disks and the kernel as files 8

2.6 Stretch A - Magic numbers 9

2.7 Stretch C - Unlink a file that is still open 9

3 Conclusions **10**

1 Introduction

Investigate file types, inode identity, links, access permissions, allocated space and the live kernel information provided by Linux.

This report follows the teacher's handout and includes every required Observe answer. Terminal-style listings replace new screenshots. Recorded output is identified with its execution environment; shortened excerpts are marked. Source files and build instructions are provided in the submission directory.

2 Tasks and results

2.1 Environment and evidence

These experiments were run on 23 October 2026 in a Debian GNU/Linux ARM64 container hosted by Docker Desktop on macOS. The commands ran as ordinary user `student`, not root, with umask `0022`. The experiment directory was a fresh temporary directory on the Linux filesystem, not the macOS shared repository. This matters for permissions, inodes, sparse files and link counts.

The terminal excerpts are real captured output; listings are shortened where indicated. `terminal-output.txt` retains the full session. The disk, memory, CPU and directory-size measurements describe the Linux environment used for validation, not a separately booted Ubuntu laptop. `collect.sh` repeats all commands without altering existing work and deletes its temporary experiment directory on exit.

2.2 Part 1 - The tree and file types

```
$ whoami
student
$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
lrwxrwxrwx 1 root root    7 Aug 24 00:00 /bin -> usr/bin
crw-rw-rw- 1 root root 1, 3 Oct 23 09:33 /dev/null
drwxr-xr-x 1 root root 4096 Oct 23 09:35 /etc
-rw-r--r-- 1 root root 1125 Oct 23 09:35 /etc/passwd
drwxrwxrwt 1 root root 4096 Oct 23 09:38 /tmp
$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, ARM aarch64, dynamically linked [...]
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin
```

2.2.1 1. Types from ls -l

Path	First character	Type
`/etc`	`d`	Directory holding system configuration.
`/etc/passwd`	`-`	Regular file with account metadata.

Path	First character	Type
/bin	l	Symbolic link to usr/bin, resolved as /usr/bin.
/dev/null	c	Character device; writes are discarded and reads return EOF.
/tmp	d	Directory for temporary files, with a sticky bit.

The first character describes the inode type, while the next nine characters describe access permissions. A symbolic link has its own inode and stores a pathname to resolve.

2.2.2 2. What /bin is

On this merged-/usr installation, /bin is a symbolic link, not a separate directory. Its stored target is the relative pathname usr/bin; relative to its parent /, that is /usr/bin.

2.2.3 3. Character and block devices

/dev/null is a character device, major 1, minor 3. lsblk identifies virtual disk vda and partition vda1; df reports the source /dev/vda1 for the files bind-mounted at /etc/hosts, /etc/hostname and /etc/resolv.conf. The isolated container does not expose that block-device node in its own /dev, so a b-type entry could not be directly listed there. This is a container visibility limit, not evidence that the Linux VM has no block devices. On a full Linux installation, ls -l /dev/vda /dev/vda1 (or the actual sda/nvme names) lists the block nodes. No synthetic block node was created for this report.

2.2.4 4. Compare /var/log and /usr

```
$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
504K    /var/log
1.5G    /usr
```

/usr is larger by about 1.5 GiB, roughly 3,100 times the rounded /var/log value. These are allocated-space estimates from du, not sums of apparent lengths. Diagnostics for unreadable paths were suppressed as the handout requests.

2.3 Part 2 - Inodes and links

2.3.1 1. Inode number and link count

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6       Blocks: 8       IO Block: 4096   regular file
Device: 0,97    Inode: 33824229    Links: 1
Access: (0644/-rw-r--r--)  Uid: (1001/student) Gid: (1001/student)
[time fields omitted]
$ ls -li notes.txt
```

```
33824229 -rw-r--r-- 1 student student 6 Oct 23 09:38 notes.txt
```

`Inode: 33824229` identifies this inode within its filesystem; `Links: 1` counts hard-link directory entries naming it. The inode is not globally unique without a device/filesystem identifier. The filename lives in a directory entry, not inside the inode. `echo hello` writes six bytes, including its newline.

2.3.2 2. Removing the original name

```
$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
33824229 -rw-r--r-- 2 student student 6 Oct 23 09:38 hard.txt
33824229 -rw-r--r-- 2 student student 6 Oct 23 09:38 notes.txt
$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
33824229 -rw-r--r-- 1 student student 6 Oct 23 09:38 hard.txt
```

Both names point to the same inode. `rm` unlinks a directory entry and decreases the inode's link count; it does not distinguish an original name from a hard link. `hard.txt` remains readable with the same inode and content. Storage is reclaimable when the final hard link and final open reference are both gone.

2.3.3 3. Dangling symbolic link and its size

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
33824229 -rw-r--r-- 1 student student 6 Oct 23 09:38 hard.txt
33824251 lrwxrwxrwx 1 student student 8 Oct 23 09:38 soft.txt -> hard.txt
$ cat soft.txt
hello
$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 student student 8 Oct 23 09:38 soft.txt -> hard.txt
```

The symlink has a different inode and stores the eight-byte string `hard.txt` (no terminating NUL or newline is counted in its size). After unlinking the target, the symlink still exists, but resolving its pathname fails. A symlink does not increase the target's hard-link count.

2.3.4 4. Directory link counts

Prediction: 2 links initially, then 4 after creating two immediate subdirectories. The measured result agrees:

```
$ mkdir d ; ls -ld d
 drwxr-xr-x 2 student student 4096 Oct 23 09:38 d
$ mkdir d/x d/y ; ls -ld d
 drwxr-xr-x 4 student student 4096 Oct 23 09:38 d
```

The parent entry naming `d` and `d/.` account for the first two links. `d/x/..` and `d/y/..` each add a reference to `d`. The entry `d/..` points to d's parent, not back to d. This common Unix rule is filesystem dependent; the experiment filesystem implements it.

2.3.5 5. Apparent size versus allocated space

```
$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-r--r-- 1 student student 2 Oct 23 09:38 one.txt
4.0K    one.txt
$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-r--r-- 1 student student 1.0G Oct 23 09:38 big.img
0       big.img
$ rm big.img
```

The two bytes are `x` and a newline. This filesystem allocates data in 4 KiB units; stat reports eight 512-byte accounting blocks. `truncate -s 1G` sets the logical length to 1,073,741,824 bytes without writing the data. The file is sparse: reads from holes return zeros and no data blocks are allocated yet. The displayed zero does not mean that its inode and other metadata require no storage.

2.4 Part 3 - Permissions

2.4.1 1. Numeric modes and uses

Mode	Nine permission bits	Typical use
`600`	`rw-----`	A private text file or private SSH key.
`644`	`rw-r-r-`	A document readable by other users but writable only by its owner.
`755`	`rwxr-xr-x`	A public directory or executable.
`700`	`rwx----`	A private directory or personal executable.

The type character shown by `ls -l` is separate from these nine bits. The owner, group and others each use weights read=4, write=2 and execute/search=1.

2.4.2 2. Why ownership does not bypass 000

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 student student 7 Oct 23 09:38 s.txt
$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied
$ chmod u+r s.txt ; cat s.txt
secret
```

For this ordinary user, the kernel selects the owner's permission class, which has no read bit under mode 000. Ownership permits changing the mode with `chmod`; it does not grant automatic data access. After adding owner-read, the file can be opened. Running as root could invalidate this experiment because privileged processes can bypass ordinary access checks.

2.4.3 3. Umask and new objects

```
$ umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
```

```
0022
-rw-r--r-- 1 student student    0 Oct 23 09:38 new.txt
drwxr-xr-x 2 student student 4096 Oct 23 09:38 newdir
```

The starting mode for an ordinary new file is 0666; for a directory it is 0777. Applying `mode & ~umask` gives `0666 & ~0022 = 0644` and `0777 & ~0022 = 0755`. A umask removes bits; it does not add executable permission to new text files. ACLs can further affect creation, but none is needed to explain these results.

2.4.4 4. Directory read versus execute

Prediction for read-only: names can be listed, but `box/f` cannot be opened. Prediction for execute-only: listing fails, but a known filename can be opened if the file itself permits reading. Both predictions were confirmed.

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied
$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi
$ chmod 755 box
```

Directory `r` allows reading its list of names. Directory `x` allows searching/resolving a known entry and traversing the directory. Reading a file needs search permission on each parent directory and read permission on the file. Directory `w` together with `x` controls creating/removing names, independently of the file's own write bit.

2.4.5 5. setuid, sticky bit and /etc/shadow

```
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 72048 Dec 14 2025 /usr/bin/passwd
drwxrwxrwt 1 root root 4096 Oct 23 09:38 /tmp
$ cat /etc/shadow
cat: /etc/shadow: Permission denied
$ ls -l /etc/shadow
-rw-r----- 1 root shadow 617 Oct 23 09:35 /etc/shadow
```

The owner's `s` denotes setuid plus owner-execute. When permitted by the execution environment, executing `passwd` gives it the effective UID of its owner, root, so it can update protected password data after enforcing authentication rules. The sticky `t` on `/tmp` permits shared creation while restricting removal/renaming to the file owner, directory owner or a privileged process. Sticky does not prevent reading someone else's readable file.

`/etc/shadow` is owned by root and group shadow; `student` cannot read it. World-readable account metadata in `/etc/passwd` is separate from protected password hashes.

2.5 Part 4 - Disks and the kernel as files

2.5.1 1. Disk drawing and the root mount

Recorded `lsblk` excerpt:

```
NAME   MAJ:MIN   SIZE RO TYPE MOUNTPOINTS
vda    254:0       1T  0 disk
`-vda1 254:1    1024G  0 part /etc/hosts
                              /etc/hostname
                              /etc/resolv.conf
vdb    254:16  536.9M  1 disk
[zero-sized network-block devices omitted]
```

The measured topology is:

```
Docker Desktop Linux VM
  vda (1 TiB virtual disk)
    vda1 (1024 GiB partition, ext4 backing storage)
      container image layers + writable layer
        overlay mounted at / inside this container
      bind-mounted configuration files (/etc/hosts, etc.)
  vdb (536.9 MiB, read-only virtual disk)
```

`mount` reports `overlay on / type overlay (rw,relatime,...)`. No partition is mounted directly on the container's `/`; overlay combines read-only image layers with a writable layer. The backing `/dev/vda1` mounts are ext4, verified with `findmnt -no SOURCE,FSTYPE,TARGET /etc/hosts`. Thus it would be incorrect to label the container root mount itself ext4. This drawing describes the VM/container topology that was actually available.

2.5.2 2. Root filesystem fullness

```
$ df -h /
Filesystem  Size  Used Avail Use% Mounted on
overlay    1007G   25G  931G   3% /
```

The root mount reports about 25 GiB used and 3% usage, with about 931 GiB available. The 1 TiB virtual disk and rounded `df` capacity differ because of filesystem metadata and accounting. Available space can also exclude reserved blocks; used plus available need not equal total.

2.5.3 3. CPU and RAM from /proc

```
$ grep -c processor /proc/cpuinfo
8
$ head -3 /proc/meminfo
MemTotal:        8025700 kB
MemFree:         1499908 kB
MemAvailable:    3892820 kB
```

There are 8 visible logical CPUs (not necessarily 8 physical cores). `MemTotal` is 8,025,700 KiB, about 7.65 GiB. In this container, these `/proc` readings describe the Linux VM's visible resources; CPU quotas and memory cgroup limits, if set, are separate from these values. `MemAvailable` estimates memory

usable without swapping, including reclaimable cache, while `MemFree` is completely unused memory.

2.5.4 4. Which program /proc/self/status describes

```
$ head -5 /proc/self/status
Name:   head
Umask:  0022
State:  R (running)
Tgid:   8764
Ngid:   0
$ ls -l /proc/self/fd
[excerpt]
0 -> /dev/null
1 -> pipe:[29578117]
2 -> pipe:[29578118]
3 -> /proc/8761/fd
```

The name is `head`: `/proc/self` resolves for the process that opens it, which is `head`, rather than the shell that launched the command. In the next command, `self` refers to `ls`, so descriptor 3 is its own open directory descriptor. Standard output/error point to capture pipes because this session was recorded. These are live kernel views, not ordinary on-disk copies of process data.

2.6 Stretch A - Magic numbers

```
$ gzip -c /etc/passwd > notes.txt
$ file notes.txt
notes.txt: gzip compressed data, was "passwd", [...] from Unix [...]
$ xxd notes.txt | head -1
00000000: 1f8b 0808 492a db6a 0003 7061 7373 7764  ....I*.j..passwd
```

The leading bytes `1f 8b` identify gzip, and `08` is its compression method (deflate). `file` examines the content signature, so the misleading `.txt` extension does not make the compressed stream plain text. Later header bytes contain flags, timestamps and the original name; they can change between runs.

2.7 Stretch C - Unlink a file that is still open

```
$ echo retained-data > bigfile
$ exec 3< bigfile
$ rm bigfile
$ ls -l /proc/$$/fd/3
lr-x------ 1 student student 64 Oct 23 09:38 /proc/8770/fd/3 -> /tmp/os-lab5.18dfip/bigfile
    (deleted)
$ cat <&3
retained-data
$ exec 3<&-
```

The directory entry has disappeared but descriptor 3 still holds the inode open, as the `(deleted)` suffix indicates. Data remains readable through that descriptor. With no remaining hard links or other open references, closing descriptor 3 permits the filesystem to reclaim the storage. This is the same name-versus-object distinction seen in the hard-link experiment.

3 Conclusions

The experiments show why names, inodes and open descriptors must be treated separately. A hard link preserves access after another name is removed, a symbolic link can dangle, and an open descriptor can retain an unlinked file. Permissions distinguish file access from directory search, while sparse files distinguish apparent size from allocation. The measured storage topology and resource values belong to the documented Linux VM/container environment.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 5: *The File System with Your Own Hands*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `f780d7850765d74cbdcd1fff4b822d8829f7c9bd`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/f780d7850765d74cbdcd1fff4b822d8829f7c9bd`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.